Arjan Khera – Security Log Analysis Tool

Chapter 1: Introduction

This tool was designed and developed as a security investigation tool to aid a security analyst working in a security operations centre (SOC). The tool aims to detect suspicious activity, investigate potential security threats and generate an investigation report based on data from a variety of provided log files. Manually parsing log files is slow and laborious, this tool automates the process to make it much quicker and more efficient. The tool detects multiple threats: malicious/suspicious PowerShell commands from a PowerShell log, brute force login attempts from an auth.log file, and suspicious access patterns in an access.log file before outputting and presenting all of the data to a generated investigation report text file.

Chapter 2: Methodology

The tool uses three main functions for different types of threat detection, investigation and reporting. Each log file was stored in a variable for use in the equivalent function.

The first function detects potentially malicious/suspicious patterns in the PowerShell log using Select-String to search for commands such as “EncodedCommand” and “ExecutionPolicy Bypass” which are unusual commands for a trusted user to input. Initially the “Invoke-WebRequest” command was also included in this threshold, however upon examination of the PowerShell log and the detection results, it was decided that this logic could result in false positives of trusted users inputting the Invoke WebRequest command to a trusted website. In order to solve this problem, a separate detection threshold was implemented in this function. The logic of this detection threshold was to flag any “Invoke WebRequest” commands that were using an insecure protocol such as HTTP. Any web requests to a trusted HTTPS domain were ignored as they are assumed to be secure and being requested by a trusted user. These suspicious PowerShell and HTTP web request alerts are then stored in a variable and counted, informing the user if any suspicious patterns are detected and printing the line of the log in which the pattern is detected.

The second function initially aimed to detect abnormal login behaviour in the auth.log file using Select-String to search for strings such as “Failed password”, “authentication failure” or simply “error”, however this resulted in thousands of matches which contained many results of normal user behaviour. This was later revised to search for “Failed password” but only where the amount of failed password attempts is five or more, which is an indicator of a potential brute force attack. The goal of this threshold is to eliminate any false positives from a trusted user simple mistyping their password or

forgetting their password. A trusted user might attempt a password two or three times before either entering the correct password or deciding to reset their password. The function then searches the resulting lines of data for the corresponding IP address for the brute force attacks and appends the IP address to a list, generating a list of the top offending IP addresses and printing the top 5 offending IP addresses to the screen for the analyst to review, along with a count of how many times a brute force attack was attempted from each IP address. The list of top IP addresses and the count of their attempts are stored in variables for output to the screen and to the investigation report. If no potential brute force threats are detected in the log file, the function prints "No abnormal login behaviour detected" to the screen instead.

The last detection function in the tool is a function to detect suspicious access patterns in the access.log file, again using Select-String to parse the log file but this time cross referencing the log with the IOC.txt file that contains a list of known malicious IP addresses and web paths. The content of the IOC.txt file is stored in a variable using Get-Content, removing any blank spaces, and the pattern of each line in the file is searched for in the access.log file. If a matching pattern is detected in the log file, the user is alerted and a sample of the first 5 matches are printed to the screen, with a counter displaying the total number of matches in the log. Both the count and the detected threats are stored in a variable for display.

The final function in the tool is simply for generating a formatted investigation report for the analyst to easily read and review after running the tool. The variables that store the alerts from each function are used to declare whether each log file has any threats ([ALERT]) or is safe any doesn't contain any potential security threats ([CLEAR]). The report is segmented with each section reporting on the relative log file, starting with the PowerShell log file, then the auth.log file and ending with the access.log file. The PowerShell log section displays the status ([ALERT] or [CLEAR]), subsequently displays the total amount of suspicious patterns or potential threats detected in the log and provides a sample of the first matching pattern. The auth log section provides the status, then shows the total potential brute force threats detected and a sample of the first result. It also then displays the aforementioned list of the top source IP addresses of the brute force attacks for a quick view of the most dangerous source IPs. Finally, the access log segment reports the total amount of suspicious patterns detected from cross referencing the IOC.txt file with the log and provides a sample matched pattern, if one exists. The code finishes with calling each function for the tool to run successfully. The code for the tool is completely self-contained, allowing the user to simply run a single command for each function to work, negating the need for scripts outside of the main logic.

Chapter 3: Example Results

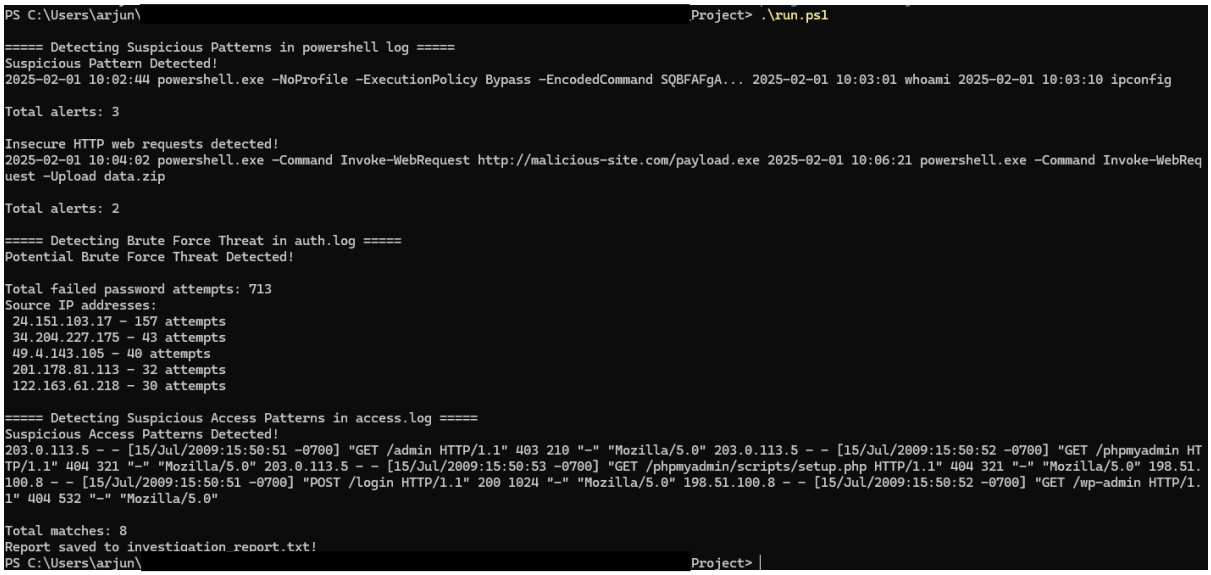


Figure 1

As seen in Figure 1, the user runs a single command to execute the tool. The tool then scans the log files using each function to extract the required data, displaying the results of each function to the screen – a total amount of alerts from each file, as well as a sample of the offending line in the log – before informing the user that a report has been saved to investigation_report.txt.

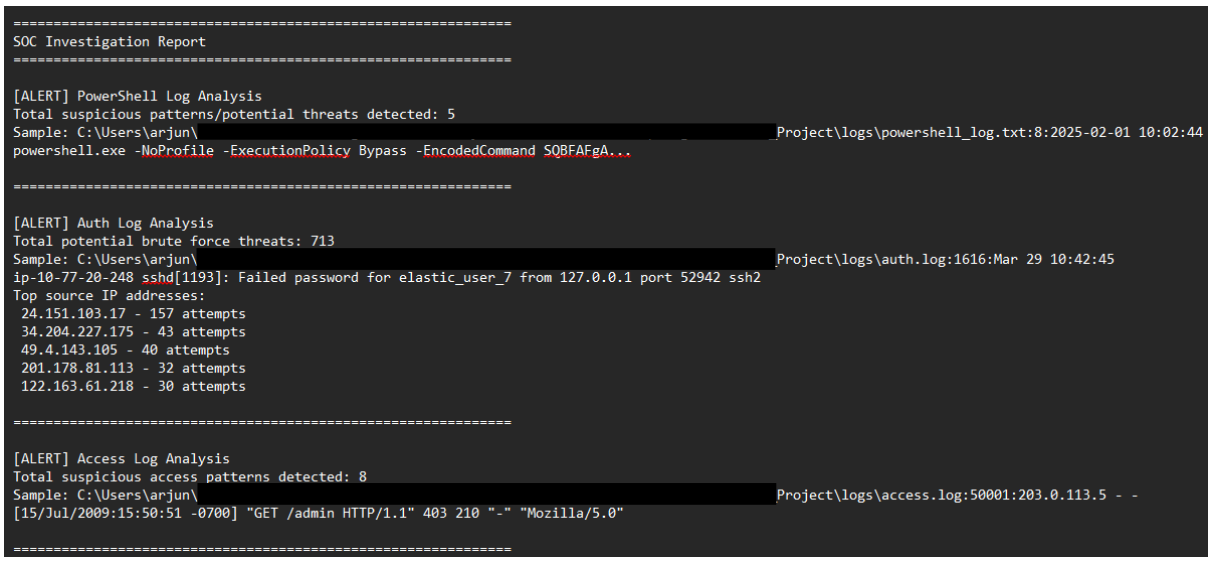


Figure 2

Figure 2 depicts the resulting SOC investigation report generated by the tool. The report is formatted and easy to read with segments for the results from each function. A total alert counter is appended to each section as well as a sample of the potential threats detected.

As a proof of concept, the suspected threat indicator lines were removed from the PowerShell log, and the tool ran again to ensure proper parsing and reporting of the log files.

```
2025-02-01 09:55:12 User login: student

# Normal activity
2025-02-01 09:57:10 powershell.exe -Command Get-Process
2025-02-01 09:58:12 powershell.exe -Command Get-Service

# Suspicious start

# Reconnaissance


# Benign web request
2025-02-01 10:03:55 powershell.exe -Command Invoke-WebRequest https://microsoft.com/update.exe

# Malicious download


# Collection


# Exfiltration


# Another user normal behaviour
2025-02-01 11:15:02 User login: admin
2025-02-01 11:16:12 powershell.exe -Command Get-EventLog
```

Figure 3

```
PS C:\Users\arjun\                                                    Project> .\run.ps1

===== Detecting Suspicious Patterns in powershell log =====
No Suspicious Patterns detected!
No insecure web requests detected!

===== Detecting Brute Force Threat in auth.log =====
Potential Brute Force Threat Detected!

Total failed password attempts: 713
Source IP addresses:
 24.151.103.17 - 157 attempts
 34.204.227.175 - 43 attempts
 49.4.143.105 - 40 attempts
 201.178.81.113 - 32 attempts
 122.163.61.218 - 30 attempts

===== Detecting Suspicious Access Patterns in access.log =====
Suspicious Access Patterns Detected!
203.0.113.5 - - [15/Jul/2009:15:50:51 -0700] "GET /admin HTTP/1.1" 403 210 "-" "Mozilla/5.0" 203.0.113.5 - - [15/Jul/2009:15:50:52 -0700] "GET /phpmyadmin HT
TP/1.1" 404 321 "-" "Mozilla/5.0" 203.0.113.5 - - [15/Jul/2009:15:50:53 -0700] "GET /phpmyadmin/scripts/setup.php HTTP/1.1" 404 321 "-" "Mozilla/5.0" 198.51.
100.8 - - [15/Jul/2009:15:50:51 -0700] "POST /login HTTP/1.1" 200 1024 "-" "Mozilla/5.0" 198.51.100.8 - - [15/Jul/2009:15:50:52 -0700] "GET /wp-admin HTTP/1.
1" 404 532 "-" "Mozilla/5.0"

Total matches: 8
Report saved to investigation_report.txt!
```

Figure 4

As can be seen in Figure 4, the tool successfully reported no alerts for the PowerShell log while the other functions still report the threats detected in the other logfiles. This is also clearly represented in the generated report.

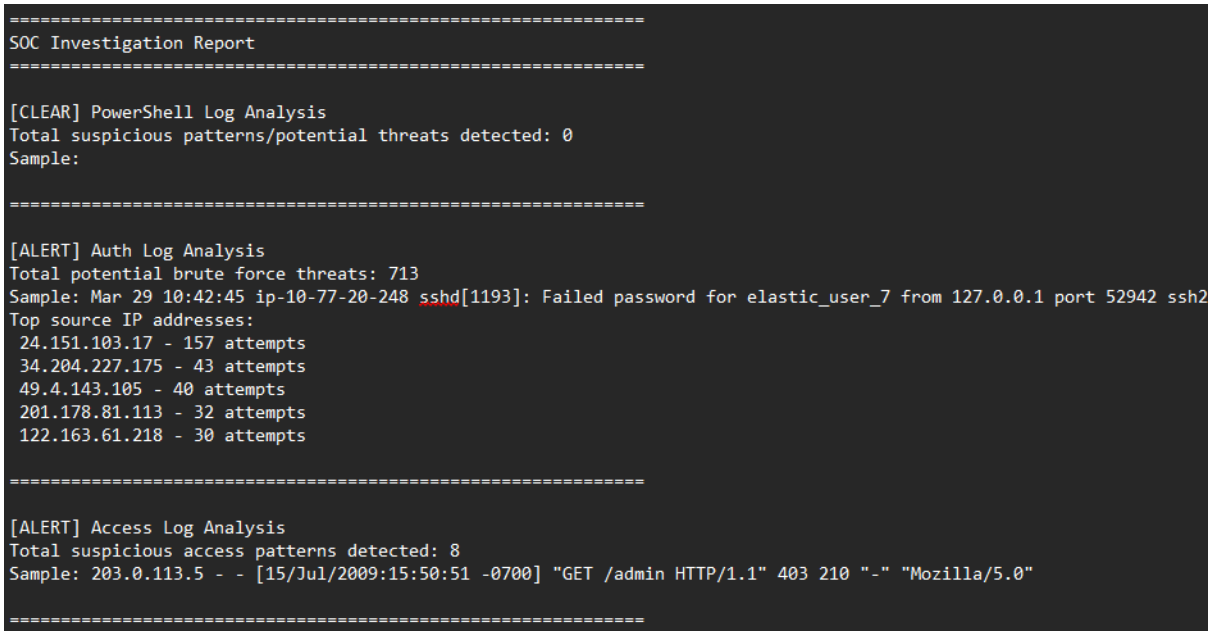


Figure 5

Chapter 4: Conclusion

Upon initial briefing and design, this tool seemed straightforward to develop and implement. From experience of threat detection and pattern matching in PowerShell, it seemed best to just use Select-String for patterns that seemed to indicate potential threats. However, after inspection of the results from this method and further scrutiny of the log files it became apparent that harsher thresholds and logic were required to acquire meaningful data from the logs through use of this tool in order to avoid problems such as false positives. The initial tool design resulted in a lot of noisy data but further reflection and refinement helped to accomplish a more robust, focused tool.

The tool is certainly limited in use, the log file names and locations are hardcoded into the tool making it far less versatile and adaptable. A potential improvement could be the prompting of the user to upload/input the location of their log file, so that it isn't necessary to place a copy of the log files into the logs folder using the correct naming format. This tool is simple to use however, as the user only needs to enter a single command to have the log files parsed, inspected and reported on. The resulting report is formatted cleanly and very easy to read, including samples for further analysis and providing a total number of threats found in each file to give the analyst an idea of how secure their systems are or if stricter controls are required.